

FerrumMD: A Gentle Introduction to Molecular Dynamics in Rust

What molecular dynamics is, what FerrumMD does, and where Rust helps

FerrumMD project overview

May 28, 2026

Who this talk is for

- People who are curious about scientific computing, Rust, or molecular simulation.
- No molecular dynamics (MD) background is assumed.
- You only need three ideas:
 - ① atoms and molecules can be modeled as particles,
 - ② particles move because forces act on them,
 - ③ a computer simulation repeats a small update many times.

What problem does molecular dynamics solve?

- Experiments can show what molecules do, but not always every tiny motion.
- MD creates a computational “movie” of atoms and molecules over time.
- Typical questions:
 - How does a liquid arrange itself?
 - Does a molecule remain stable?
 - How do temperature, pressure, or force-field choices affect behavior?
- FerrumMD is a Rust codebase for building and studying these simulation workflows.

MD in one minute

Basic idea

Start with positions and velocities, compute forces, then move everything a tiny bit.

- ① Read or build a molecular system.
- ② Compute forces from the current geometry.
- ③ Update positions and velocities using Newton's laws.
- ④ Keep particles inside the simulation box and apply controls.
- ⑤ Save energies, temperatures, and trajectory frames.

state $\rightarrow$ forces $\rightarrow$ updated state $\rightarrow$ repeat

A few useful words

Particle An atom or coarse-grained bead with a position, velocity, and mass.

Force field Parameters and formulas that approximate molecular forces.

Integrator The numerical method that advances the simulation clock.

Trajectory The saved sequence of molecular snapshots.

Ensemble The thermodynamic conditions being approximated, such as constant energy or temperature.

Why this project matters

- **Learning value:** FerrumMD exposes the pieces of an MD engine in a readable systems language.
- **Engineering value:** Rust's ownership model helps organize mutable simulation state safely.
- **Scientific value:** explicit units, tests, and reproducible workflows are central to trustworthy simulations.
- **Scope today:** the main building blocks rather than production-scale chemistry.

Simulation core

- particle and molecule state
- simulation box and periodic boundaries
- Lennard–Jones and bonded forces
- velocity-Verlet time stepping

Workflow support

- temperature and pressure controls
- input/output for common file formats
- example water-like systems
- optional viewer, Python, and MPI paths

The simulation state: what the program stores

- **Coordinates:** where each particle is in the box.
- **Velocities:** how fast each particle is moving.
- **Forces:** the current push or pull on each particle.
- **Parameters:** masses, charges, bond constants, and interaction strengths.
- **Box:** the finite volume that stands in for a larger material.

Unit convention

FerrumMD keeps units explicit: nm, ps, amu, and kJ/mol.

Forces: how particles influence each other

Nonbonded interactions

- Lennard–Jones models short-range attraction and repulsion.
- Periodic boundaries make the small box behave more like bulk matter.
- Cell subdivision reduces the number of pair checks.

Bonded interactions

- Harmonic bonds act like springs between connected atoms.
- Small molecular examples make the math easier to inspect.
- Topology readers help import force-field definitions.

Time stepping: turning forces into motion

- FerrumMD uses velocity-Verlet as a central integration method.
- Each step is tiny, so many steps are needed for a useful trajectory.
- The integrator must balance speed, stability, and energy behavior.
- Initialization can sample velocities from a Maxwell–Boltzmann distribution.

Beginner mental model

It is like a physics game loop, but numerical accuracy and units matter much more.

Temperature, pressure, and ensembles

- Real experiments often control temperature or pressure.
- MD approximates these conditions with *ensembles*:
 - NVE: constant particles, volume, and energy,
 - NVT: constant particles, volume, and temperature,
 - NPT: constant particles, pressure, and temperature.
- FerrumMD includes thermostat and barostat paths, including Berendsen and Nose–Hoover thermostats.

Inputs, outputs, and visualization

- Read starting structures and parameters from familiar molecular formats.
- Supported inputs include PDB, GRO, LAMMPS data/dump, Martini, and CHARMM-style files.
- Write GRO/XTC outputs so trajectories can be viewed in external tools such as VMD.
- Optional Rust viewer and Python bindings provide extension points for demos and analysis.

The honest Rust pitch

Not the claim

Rust does *not* make an MD engine automatically faster than C++. Well-written C++ MD engines can be extremely fast.

The stronger claim

Rust offers C/C++-level native performance while making many dangerous scientific-code bugs harder to write.

- The goal is not to beat mature engines just by choosing Rust.
- The goal is a safe, modular, maintainable path from research prototype to serious simulation software.

Rust advantage 1: memory safety without GC

- MD engines manipulate large arrays of positions, velocities, forces, neighbor lists, cell lists, bonds, and topology.
- C/C++ bugs can include dangling pointers, use-after-free, double free, out-of-bounds access, and invalidated references.
- Rust's ownership and borrowing rules catch many of these hazards at compile time.
- Example MD hazard: mutating a particle array while another part of the code still holds references into it.

Why this matters

Force loops, neighbor-list builds, integration, and boundary-condition updates are all mutation-heavy and aliasing-sensitive.

Rust advantage 2: safer parallelism

- MD wants parallelism in pair forces, domain decomposition, neighbor-list builds, reductions, and trajectory analysis.
- The danger is two threads accidentally writing the same force entry or sharing mutable state unsafely.
- Rust forces shared mutation to be explicit: locks, atomics, chunked mutable slices, or safe parallel iterators.
- Good MD designs usually avoid “mutex everywhere”:
 - each worker owns a particle chunk, or
 - each worker writes to its own force buffer, then buffers are reduced safely.

Rust advantage 3: encode simulation meaning in types

- An MD engine is not just math; it is a large state machine.
- State includes box dimensions, cutoffs, thermostat/barostat state, masses, topology, random-number generators, and step counters.
- Rust types can make invalid states harder to represent:
 - Positions, Velocities, Forces, and Masses instead of anonymous arrays,
 - enums for algorithm choices such as no thermostat, Andersen, Berendsen, or Nose–Hoover,
 - separate immutable parameters from mutable evolving state.
- The compiler becomes a strict reviewer when ownership or invariants change.

Rust advantage 4: tooling and modular architecture

Cargo workflow

- `cargo build`
- `cargo test`
- `cargo bench`
- `cargo doc`
- `cargo fmt` and `cargo clippy`

Engine design

- traits for force fields, integrators, and thermostats
- feature flags for MPI, viewer, Python, or future GPU paths
- unit tests for forces and regression tests for energy behavior
- clean APIs instead of one giant procedural script

Rust advantage 5: FFI and research workflows

- Rust can interoperate with C, C++, Python, MPI libraries, and existing visualization or analysis tools.
- A practical architecture is:
 - Rust core engine for simulation kernels,
 - Python interface for setup, analysis, and experimentation,
 - file-format compatibility for tools such as VMD.
- This supports a research workflow: prototype quickly, test carefully, and keep the performance-critical core compiled and typed.

Where C++ still has advantages

- C++ has a larger and older HPC ecosystem.
- CUDA/HIP and GPU tooling are more mature in C++ workflows.
- Many scientific developers already know C++.
- Existing engines such as GROMACS, LAMMPS, NAMD, AMBER, and OpenMM have decades of optimization and validation.

Positioning FerrumMD

FerrumMD should be pitched as a modern, safe, modular research engine—not as faster than mature production engines simply because it uses Rust.

How to read the codebase as a newcomer

- ① Start with a runnable example to see the input, loop, and output flow.
- ② Inspect the state and molecule modules to understand stored data.
- ③ Follow one force calculation from inputs to resulting forces and energy.
- ④ Then look at integrators, ensembles, and I/O formats.

Tip

You do not need to understand every force-field detail before contributing to tooling, tests, examples, or documentation.

Roadmap and open opportunities

- Rigid-water constraints such as SETTLE or SHAKE/RATTLE.
- Topology-correct nonbonded exclusions and special pairs.
- Electrostatics and units audits for stronger TIP3P fidelity.
- Rust-focused opportunities:
 - clearer examples and validation tests,
 - SIMD or GPU experiments,
 - stronger parallel decomposition beyond demo MPI,
 - property-based tests for physical invariants.

Takeaways

- ① MD is a repeated loop: compute forces, update motion, save observations.
- ② FerrumMD provides an approachable Rust implementation of that loop and its surrounding workflow.
- ③ New contributors can start with examples, units, I/O, tests, and documentation before tackling advanced physics.

Questions & discussion